# AEM HTL Blocks

Complete reference — core blocks, advanced blocks, expression options, and global objects

---

## Core HTL Blocks

### [Core] data-sly-use

Instantiates a Use-object (Sling Model, WCMUsePojo, or JS script) and binds it to a variable. This is the primary bridge between HTL templates and Java backend logic. Multiple use-objects can be bound in a single element.

**SYNTAX**

```
<!-- Java Sling Model -->
<div data-sly-use.model="com.example.MyModel">
<p>${model.title}</p>
<p>${model.description}</p>
</div>

<!-- JS use script -->
<div data-sly-use.logic="logic.js">
<span>${logic.greeting}</span>
</div>
```

**EXAMPLE OUTPUT**

```
<div>
<p>Welcome to AEM</p>
<p>Sample component</p>
</div>
```

Notes: Host tag is kept. Works with Sling Models (most common), WCMUsePojo, or .js scripts.

---

### [Core] data-sly-text

Sets the text content of an element, replacing its existing children. All output is HTML-escaped by default (XSS-safe). Use @ context='html' to render raw HTML (e.g. RTE fields).

**SYNTAX**

```
<!-- Replaces inner content (HTML-escaped) -->
<p data-sly-text="${properties.jcr:title}">
Placeholder text
</p>

<!-- Equivalent inline expression -->
<h1>${properties.jcr:title}</h1>

<!-- Raw HTML (use carefully) -->
<div data-sly-text="${richText @ context='html'}"></div>
```

**EXAMPLE OUTPUT**

```
<p>My Page Title</p>
<h1>My Page Title</h1>
```

Notes: The inline ${...} syntax is shorthand equivalent. Never use context='html' with untrusted content.

---

## [Core] data-sly-attribute

Sets or removes HTML attributes dynamically. If the expression evaluates to false, null, or empty, the attribute is completely omitted from output. You can target a single attribute or pass a map object.

**SYNTAX**

```
<!-- Single attributes -->
<a data-sly-attribute.href="${linkModel.url}"
data-sly-attribute.title="${linkModel.label}">
Click here
</a>

<!-- Conditional: omitted when false -->
<input data-sly-attribute.disabled="${isDisabled}"
type="text"/>

<!-- Map of attributes -->
<div data-sly-attribute="${attrMap}"></div>
```

**EXAMPLE OUTPUT**

```
<a href="/content/page.html" title="Go to Page">Click here</a>
<input disabled type="text"/>
```

Notes: Attribute is completely absent from HTML when value is null/false — not just empty string.

---

## [Core] data-sly-test

Conditionally includes or removes an element and all its children. When the expression is falsy, the element is completely absent from output — not hidden with CSS. Optionally bind the result to a variable for reuse (else pattern).

**SYNTAX**

```
<!-- Basic condition -->
<p data-sly-test="${properties.description}">
${properties.description}
</p>

<!-- Bind to variable for reuse -->
<div data-sly-test.hasTitle="${properties.jcr:title}">
<h1>${properties.jcr:title}</h1>
</div>

<!-- Else pattern -->
<p data-sly-test="${!hasTitle}">No title set</p>
```

**EXAMPLE OUTPUT**

```
<!-- <p> omitted (description empty) -->
<div><h1>Hello</h1></div>
<!-- <p>No title set</p> omitted (hasTitle truthy) -->
```

Notes: Binding to a variable (data-sly-test.varName) is the HTL equivalent of an else branch.

---

## [Core] data-sly-list

Iterates over a collection (List, array, Map). The host element is kept once; its children are repeated for each item. A loop metadata object (default name: itemList) provides index, count, first, last, odd, even, total.

**SYNTAX**

```
<!-- Custom variable + loop metadata -->
<ul>
<li data-sly-list.page="${model.pages}">
<span>${pageList.count}.</span>
<a href="${page.path}">${page.title}</a>
<em data-sly-test="${pageList.first}"> (Featured)</em>
</li>
</ul>
```

**EXAMPLE OUTPUT**

```
<ul>
<li><span>1.</span> <a href="/home">Home</a> <em>(Featured)</em></li>
<li><span>2.</span> <a href="/about">About</a></li>
</ul>
```

Notes: Loop metadata object name = itemName + 'List'. Properties: index (0-based), count (1-based), first, last, odd, even, total.

---

## [Core] data-sly-repeat

Like data-sly-list, but the host element itself is repeated for each item — not just its children. Essential for generating multiple sibling elements like rows or adjacent s without an extra wrapper.

**SYNTAX**

```
<!-- Host <tr> repeats per row (not just children) -->
<tr data-sly-repeat.row="${model.rows}"
class="${rowList.odd ? 'odd' : 'even'}">
<td>${row.name}</td>
<td>${row.value}</td>
</tr>
```

**EXAMPLE OUTPUT**

```
<tr class="odd"><td>Row A</td><td>100</td></tr>
<tr class="even"><td>Row B</td><td>200</td></tr>
```

Notes: data-sly-list: host kept once, children repeat. data-sly-repeat: host element itself repeats. Use repeat for , /, sibling elements.

---

## [Core] data-sly-include

Includes another HTL script file in place of the host element. The host tag is removed and replaced by the included file's output. The included script shares the same variable scope as the parent.

**SYNTAX**

```
<!-- Include a partial (host tag removed) -->
<div data-sly-include="header.html"></div>

<!-- Dynamic path -->
<sly data-sly-include="${'/apps/myapp/nav/nav.html'}"></sly>

<!-- With a use-object in scope -->
<sly data-sly-use.pojo="com.example.BreadcrumbModel"
data-sly-include="breadcrumb.html"></sly>
```

**EXAMPLE OUTPUT**

```
<!-- host tag gone, replaced by header.html output -->
```

Notes: data-sly-include = include an HTL script (same request context, variables shared). data-sly-resource = render a Sling node (new pipeline).

---

## [Core] data-sly-resource

Renders a Sling resource by dispatching through the full Sling rendering pipeline. AEM resolves the resource type and invokes its component script. This is how page components include child components (image, text, parsys).

**SYNTAX**

```
<!-- Child node by relative path -->
<div data-sly-resource="image"></div>

<!-- Override resource type -->
<div data-sly-resource="${'jcr:content/hero' @
resourceType='myapp/components/hero'}"></div>

<!-- Custom decoration wrapper -->
<div class="hero-wrapper"
data-sly-resource="${'heroImage' @
resourceType='foundation/components/image'}">
</div>
```

**EXAMPLE OUTPUT**

```
<div class="hero-wrapper">
<!-- full rendered output of image component -->
<img src="/content/dam/photo.jpg" alt="Hero"/>
</div>
```

Notes: Host tag is kept as the decoration wrapper. Use @ decoration=false to suppress AEM's component decoration div.

---

## [Core] data-sly-template + data-sly-call

data-sly-template declares a named reusable block (like a function with typed parameters). data-sly-call invokes it, passing arguments. The template element itself is not rendered on definition. Templates can be called from other files by loading them via data-sly-use.

**SYNTAX**

```
<!-- Define template -->
<template data-sly-template.card="${@ title, url, date}">
<div class="card">
<h3>${title}</h3>
<span>${date}</span>
<a href="${url}">Read more</a>
</div>
</template>

<!-- Call in a loop -->
<div data-sly-list="${articles}">
<div data-sly-call="${card @
title=item.title, url=item.path, date=item.date}"></div>
</div>

<!-- Call from another file -->
<sly data-sly-use.tmpl="templates/cards.html"
data-sly-call="${tmpl.card @ title='Hi'}"></sly>
```

**EXAMPLE OUTPUT**

```
<div class="card"><h3>Article One</h3><span>Jan 2024</span><a href="...">Read
more</a></div>
<div class="card"><h3>Article Two</h3><span>Feb 2024</span><a href="...">Read
more</a></div>
```

Notes: The data-sly-call host tag is removed. data-sly-template host tag is removed on definition. Eliminates repeated markup within or across files.

---

## [Core] tag — invisible wrapper

A special non-rendered element that is always stripped from output. Use it when you need to apply an HTL block statement (test, list, include, use) without emitting any wrapper element in the HTML.

**SYNTAX**

```
<!-- Conditional block without wrapper div -->
<sly data-sly-test="${model.hasItems}">
<ul>
<li data-sly-list="${model.items}">${item}</li>
</ul>
</sly>

<!-- Include without extra tag -->
<sly data-sly-include="footer.html"></sly>
```

**EXAMPLE OUTPUT**

```
<!-- <sly> completely gone -->
<ul><li>X</li><li>Y</li></ul>
```

Notes: The tag never appears in rendered HTML under any circumstances. It is purely an HTL authoring construct.

---

# Advanced HTL Blocks

---

## [Advanced] data-sly-unwrap

Removes the host element tag but keeps its children in place. When the expression is true (or the attribute is present without a value), the tag is stripped. When false, the tag is preserved. Classic use: conditional anchor tag — unwrap when there is no URL.

**SYNTAX**

```
<!-- Always unwrap: strips <div>, keeps children -->
<div data-sly-unwrap>
<p>This paragraph stays</p>
</div>

<!-- Conditional unwrap -->
<a href="${link.url}"
data-sly-unwrap="${!link.hasLink}">
<span>Label Text</span>
</a>
```

**EXAMPLE OUTPUT**

```
<!-- link.hasLink=true --> <a href="/page.html"><span>Label Text</span></a>
<!-- link.hasLink=false --> <span>Label Text</span>
```

Notes: data-sly-unwrap (keep children, remove tag) is the opposite of data-sly-test (remove children and tag).

---

## [Advanced] data-sly-element

Dynamically changes the tag name of the host element while keeping all its children and attributes. Allows semantically correct headings (h1–h6) driven by author dialog without writing six separate conditional blocks.

**SYNTAX**

```
<!-- Dynamic heading level from component dialog -->
<h1 data-sly-element="${properties.headingLevel || 'h2'}">
${properties.title}
</h1>

<!-- headingLevel="h3" → renders as <h3> -->
```

**EXAMPLE OUTPUT**

```
<h3>My Title</h3>
```

Notes: Without data-sly-element you would need six data-sly-test blocks (one per heading level). All existing attributes are preserved on the renamed tag.

---

## [Advanced] Expression options: @ context, @ i18n, @ join, @ format

HTL expressions support inline options after @ to control XSS escaping context, internationalisation, array joining, string formatting, and URL building. These are part of the Expression Language and combine with any block statement.

**SYNTAX**

```
<!-- context: override escaping mode -->
<div>${richText @ context='html'}</div>
<a href="${link @ context='uri'}">...</a>
<script>var x = ${val @ context='scriptString'};</script>

<!-- i18n: translate string -->
<p>${'Submit' @ i18n}</p>
<p>${'Hello {0}' @ i18n, format=[user.name]}</p>

<!-- join: render array as delimited string -->
<p>${tags @ join=', '}</p>

<!-- format: string interpolation -->
<p>${'Page {0} of {1}' @ format=[current, total]}</p>

<!-- URL building options -->
<a href="${page.path @ extension='html', selectors='print'}">
Print</a>
```

**EXAMPLE OUTPUT**

```
<p>Submit</p> <!-- translated -->
<p>Hello Alice</p>
<p>AEM, HTL, Sling</p>
<p>Page 3 of 10</p>
```

Notes: Context modes: html, text (default), uri, attributeName, elementName, scriptString, scriptToken, styleToken, unsafe (never use). URL options: extension, selectors, suffix, prependPath, appendPath, query.

---

### [Advanced] wcmmode — edit/preview/publish branching

AEM injects a wcmmode global object into every HTL template. Branch on edit, preview, disabled, or design mode to show author placeholders that are invisible on publish. Also control component decoration via data-sly-resource options.

**SYNTAX**

```
<!-- Author placeholder when no image configured -->
<div data-sly-test="${wcmmode.edit && !properties.fileReference}"
class="cq-placeholder">
Please configure an image.
</div>

<!-- Suppress decoration div entirely -->
<div data-sly-resource="${'image' @ decoration=false}">
</div>

<!-- Custom decoration tag + class -->
<div data-sly-resource="${'text' @
decorationTagName='article',
cssClassName='text-component'}">
</div>
```

**EXAMPLE OUTPUT**

```
<!-- in edit mode, no fileReference -->
<div class="cq-placeholder">Please configure an image.</div>
<!-- in publish mode: element omitted -->
<article class="text-component">...</article>
```

Notes: wcmmode properties: edit, preview, disabled (no AEM overhead), design, touchUIMode. All boolean.

---

## HTL Global Objects

Available in every HTL template without data-sly-use. Auto-adapted to light/dark mode in the IDE; always present in the AEM rendering context.

**COMMON GLOBAL OBJECTS**

```
<!-- Component node properties -->
${properties['jcr:title']}
${properties.fileReference}

<!-- Current page -->
${pageProperties['jcr:title']}
${currentPage.path}
${currentPage.parent.title}

<!-- Request -->
${request.locale}
${request.parameterMap['q']}

<!-- Sling resource -->
${resource.path}
${resource.resourceType}

<!-- i18n shorthand -->
${'Cancel' @ i18n}
```

| Object | Type | Common use |
|---|---|---|
| `properties` | ValueMap | Current component node JCR properties |
| `pageProperties` | ValueMap | Current page jcr:content properties |
| `inheritedPageProperties` | ValueMap | Merged props up the page hierarchy |
| `currentPage` | Page | AEM Page API (path, title, children…) |
| `currentNode` | Node | Raw JCR Node |
| `resource` | Resource | Sling Resource (path, resourceType…) |
| `resourcePage` | Page | Page containing the current resource |
| `request` | SlingHttpServletRequest | HTTP request, params, locale, session |
| `response` | SlingHttpServletResponse | HTTP response (rare in HTL) |
| `sling` | SlingScriptHelper | OSGi service access |
| `resolver` | ResourceResolver | Sling resource resolver |
| `wcmmode` | WCMMode | Edit/preview/publish/design flags |
| `currentDesign` | Design | Static-template design (legacy) |
| `log` | Logger | SLF4J logger for debug output |

## Quick Reference — All Blocks at a Glance

| Block | Removes host tag? | Primary purpose |
|---|---|---|
| `data-sly-use` | No | Bind Java/JS model to a variable |
| `data-sly-text` | No (replaces children) | Set text content, HTML-escaped |
| `data-sly-attribute` | No | Set/remove HTML attributes |
| `data-sly-test` | Yes (when false) | Conditionally include/remove element |
| `data-sly-list` | No (host kept once) | Repeat children over collection |
| `data-sly-repeat` | No (host repeats) | Repeat host element over collection |
| `data-sly-include` | Yes | Include another HTL script file |
| `data-sly-resource` | No | Render Sling resource via pipeline |
| `data-sly-template` | Yes (on definition) | Declare named reusable template |
| `data-sly-call` | Yes | Invoke a data-sly-template |
| `<sly>` | Always | Invisible logic wrapper |
| `data-sly-unwrap` | Yes (conditionally) | Strip host tag, keep children |
| `data-sly-element` | No (renames it) | Dynamically change tag name |

Tip: Combine multiple block statements on one element (e.g. data-sly-use + data-sly-test + data-sly-list). Evaluation order: use → test → list/repeat → include → resource → template → call → unwrap → element → attribute → text.